

Chapter 4 – Finite Fields

Introduction

- ▶ will now introduce finite fields
- ▶ of increasing importance in cryptography
 - ▶ AES, Elliptic Curve, IDEA, Public Key
- ▶ concern operations on “numbers”
 - ▶ where what constitutes a “number” and the type of operations varies considerably
- ▶ start with concepts of groups, rings, fields from abstract algebra

Group

- ▶ a set of elements or “numbers”
- ▶ with some operation whose result is also in the set (closure)
- ▶ obeys:
 - ▶ associative law: $(a.b).c = a.(b.c)$
 - ▶ has identity e : $e.a = a.e = a$
 - ▶ has inverses a^{-1} : $a.a^{-1} = e$
- ▶ if commutative $a.b = b.a$
 - ▶ then forms an **abelian group**

Cyclic Group

- ▶ define **exponentiation** as repeated application of operator
 - ▶ example: $a^3 = a \cdot a \cdot a$
- ▶ and let identity be: $e = a^0$
- ▶ a group is cyclic if every element is a power of some fixed element
 - ▶ $\forall b, b = a^k$ for some a and every b in group
- ▶ a is said to be a generator of the group

Ring

- ▶ a set of “numbers”
- ▶ with two operations (addition and multiplication) which form:
- ▶ an abelian group with addition operation
- ▶ and multiplication:
 - ▶ has closure
 - ▶ is associative
 - ▶ distributive over addition: $a(b+c) = ab + ac$

Field

- ▶ a set of numbers
- ▶ with two operations which form:
 - ▶ abelian group for addition
 - ▶ abelian group for multiplication (ignoring 0)
 - ▶ Ring
 - ▶ Division is defined with the following rule: $a/b = a (b^{-1})$.
- ▶ have hierarchy with more axioms/laws
 - ▶ group $\rightarrow$ ring $\rightarrow$ field

Modular Arithmetic Operations

- ▶ is 'clock arithmetic'
- ▶ uses a finite number of values, and loops back from either end
- ▶ modular arithmetic is when do addition & multiplication and modulo reduce answer
- ▶ can do reduction at any point, ie

$$a+b \bmod n = [a \bmod n + b \bmod n] \bmod n$$

Modulo 8 Addition Example

+	0	1	2	3	4	5	6	7
0	0	1	2	3	4	5	6	7
1	1	2	3	4	5	6	7	0
2	2	3	4	5	6	7	0	1
3	3	4	5	6	7	0	1	2
4	4	5	6	7	0	1	2	3
5	5	6	7	0	1	2	3	4
6	6	7	0	1	2	3	4	5
7	7	0	1	2	3	4	5	6

Greatest Common Divisor (GCD)

- ▶ a common problem in number theory
- ▶ $\text{GCD}(a,b)$ of a and b is the largest number that divides evenly into both a and b
 - ▶ eg $\text{GCD}(60,24) = 12$
- ▶ often want **no common factors** (except 1) and hence numbers are **relatively prime**
 - ▶ eg $\text{GCD}(8,15) = 1$
 - ▶ hence 8 & 15 are relatively prime



Chapter 6 – Contemporary Symmetric Ciphers

Possible Techniques for Improving DES

- DES is vulnerable nowadays to a brute-force attack
- It is replaced as a standard by AES
- There has been interest to provide another algorithm during the
- transition to AES
- Also interest to preserve the existing investment in software and
- hardware, increasing the security
- **Solution: iterated version of DES** Multiple enciphering with DES

Advanced Encryption Standard

- ▶ Proposed successor to DES
- ▶ DES drawbacks
 - ▶ algorithm designed for 1970s hardware implementation
 - ▶ performs sluggishly in software implementations
 - ▶ 3DES is 3 times slower due to 3 rounds
 - ▶ 64 bit blocksize needs to be increased to speed things up
- ▶ AES Overview
 - ▶ 128, 192, 256 bit blocksize (128 bit likely to be most common)
 - ▶ Not a Feistel structure, process entire block in parallel
 - ▶ 128 bit key, expanded into 44, 32bit words with 4 words used for each round

Obvious solution: double DES

- Use two keys K_1 , K_2 and encrypt the text using the two keys

$$C = E_{K_2}(E_{K_1}(P))$$

- To decrypt simply use DES decryption twice

$$P = D_{K_1}(D_{K_2}(C))$$

- The scheme involves now a key of 112 bits which should make it much more secure than DES, at least in principle

Meet-in-the-Middle attack on 2DES

- Due to Diffie, Hellman (1977)

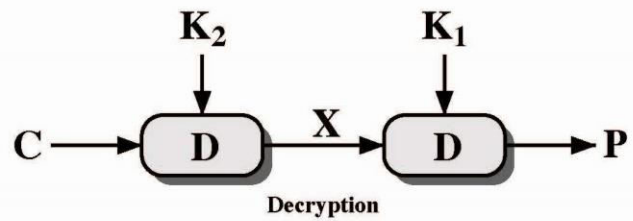
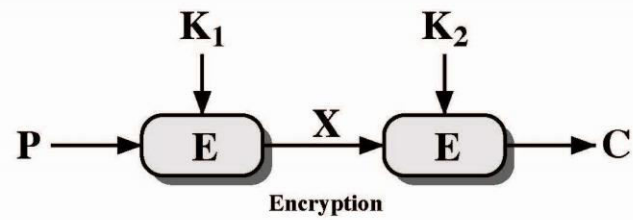
Main observation: if $C = E_{K_2}(E_{K_1}(P))$, then

$$X = E_{K_1}(P) = D_{K_2}(C)$$

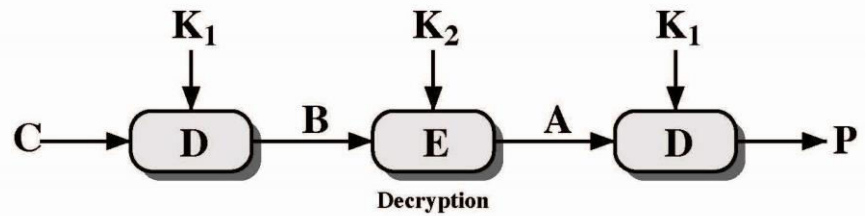
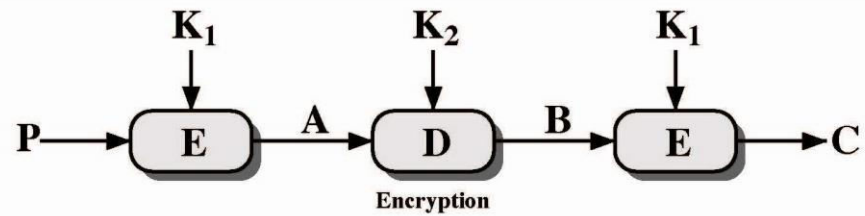
- Assume we have two pairs of plaintext-ciphertext (of only one byte each)
- Given a known pair (P, C) attack as follows:
 - Encrypt P for all 2^{56} possible keys K_1
 - Store the results in a table and sort the table by the values of X
 - Decrypt C using all possible 2^{56} possible keys K_2
 - For each decryption check the result in the table
 - In case of match, either we have the answer, or a false positive
 - Test the two keys with the second pair of plaintext-ciphertext: if they match, the correct keys were found

Triple DES

- ▶ A replacement for DES was needed
 - ▶ theoretical attacks that can break it
 - ▶ demonstrated exhaustive key search attacks
- ▶ AES is a new cipher alternative
- ▶ Before AES alternative
 - ▶ use multiple encryptions with DES
- ▶ Triple-DES is the chosen form



(a) Double Encryption



(b) Triple Encryption

Figure 6.1 Multiple Encryption

Why Triple-DES?

- ▶ why not Double-DES?
 - ▶ NOT same as some other single-DES use, but have
- ▶ meet-in-the-middle attack
 - ▶ works whenever use a cipher twice
 - ▶ since $X = E_{K1}[P] = D_{K2}[C]$
 - ▶ attack by encrypting P with all keys and store
 - ▶ then decrypt C with keys and match X value
 - ▶ can show takes $O(2^{56})$ steps

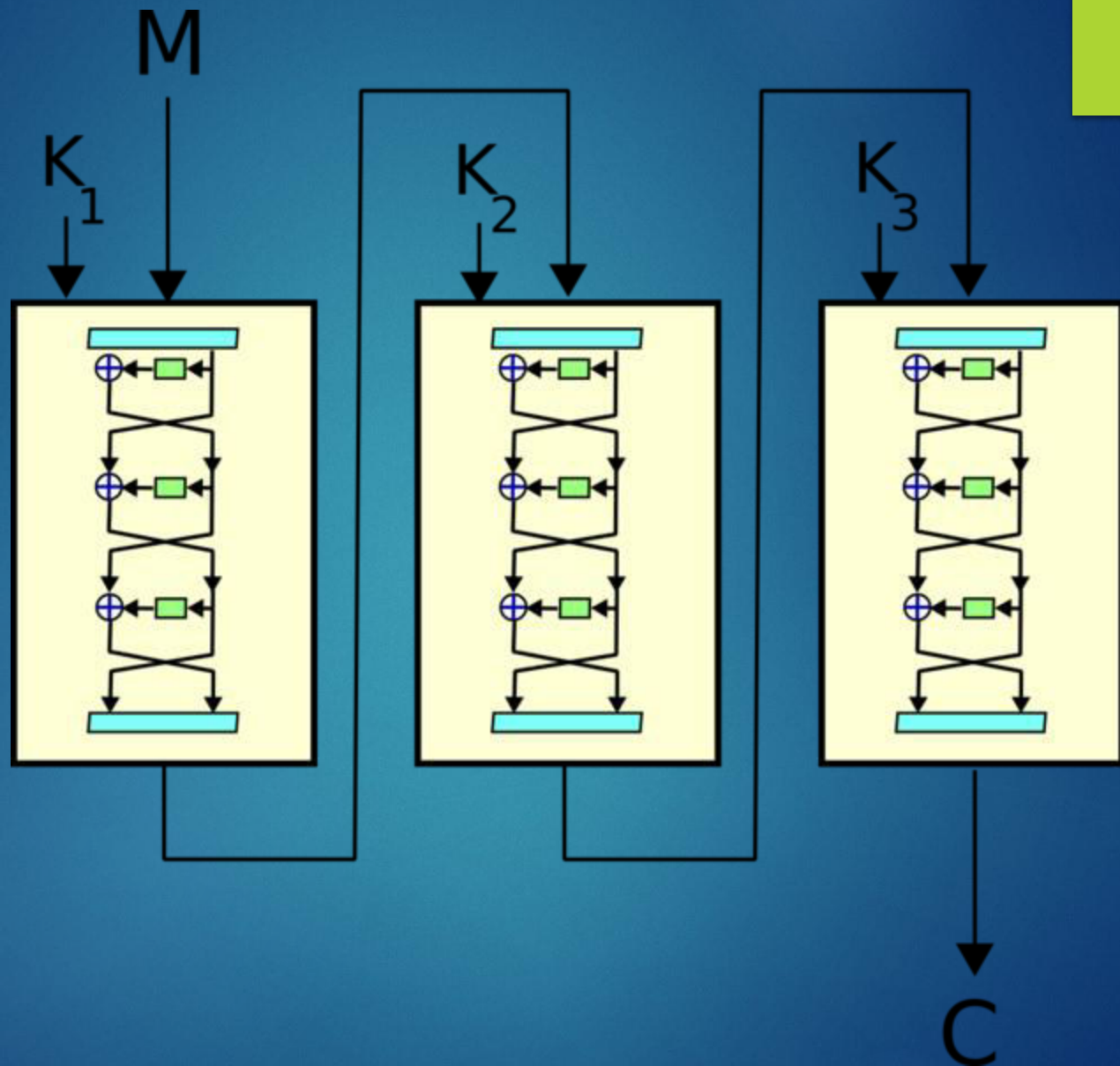
Triple-DES with Two-Keys

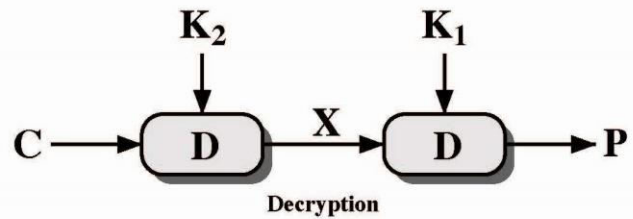
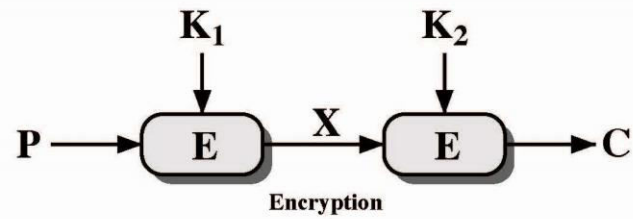
- ▶ hence must use 3 encryptions
 - ▶ would seem to need 3 distinct keys
 - ▶ Key of $56 \times 3 = 168$ bits seems too long
- ▶ but can use 2 keys with E-D-E sequence
 - ▶ $C = E_{K1}[D_{K2}[E_{K1}[P]]]$
 - ▶ No cryptographic significance to the use of D in the second step
- ▶ standardized in ANSI X9.17 & ISO8732
- ▶ no current known practical attacks
 - ▶ some are now adopting Triple-DES with three keys for greater security

Triple-DES with Three-Keys

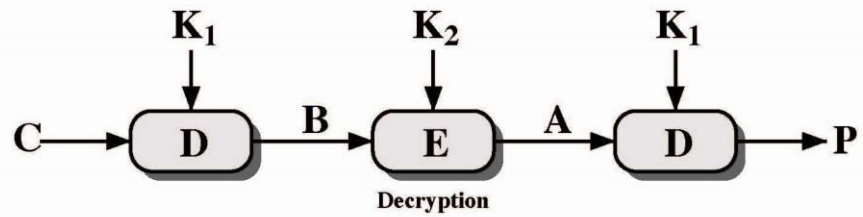
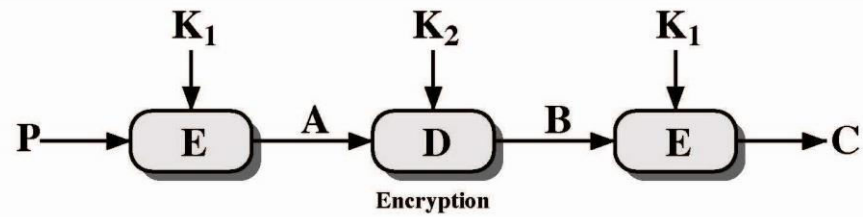
- ▶ although there are no practical attacks on two-key Triple-DES, there are some indications
- ▶ can use Triple-DES with Three-Keys to avoid even these
 - ▶ $C = E_{K3}[D_{K2}[E_{K1}[P]]]$
- ▶ has been adopted by some Internet applications

3DES





(a) Double Encryption



(b) Triple Encryption

Figure 6.1 Multiple Encryption

Block/Stream Ciphers

- Stream ciphers
 - For applications that require encrypt/decrypt of a stream of data
 - Examples: data communication channel, browser/web link
- Block ciphers
 - For applications dealing with blocks of data
 - Examples: file transfer, e-mail, database
- Either type can be used in virtually any application



Network Security

Chapter 6 – “Block Cipher Operation and Stream Ciphers”

Block/Stream Ciphers

- Stream ciphers
 - For applications that require encrypt/decrypt of a stream of data
 - Examples: data communication channel, browser/web link
- Block ciphers
 - For applications dealing with blocks of data
 - Examples: file transfer, e-mail, database
- Either type can be used in virtually any application

Stream cipher structure

- Process the message byte by byte (as a stream)
- Typically have a (pseudo) random **stream key** that is XORed with plaintext bit by bit
- Randomness of **stream key** completely destroys any statistical properties in the message
 - $C_i = M_i \text{ XOR } \text{StreamKey}_i$
- The simplest encryption/decryption algorithm possible!
- A stream cipher is similar to the one-time pad discussed a few lectures back
 - The difference is that a one-time pad uses a genuine random number stream, whereas a stream cipher uses a pseudorandom number stream generated based on a secret key
- One must never reuse stream key
 - Otherwise can remove effect and recover messages
 - XOR two ciphertexts obtained with the same key stream to obtain the XOR of the plaintexts – enough to know about the structure of the files to effectively attack them

Stream cipher structure

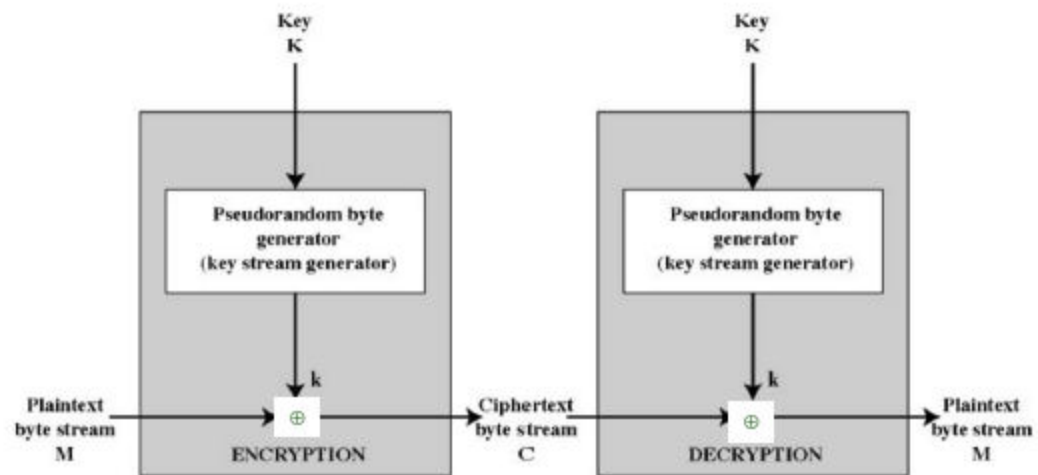


Figure 6.8 Stream Cipher Diagram

Pseudorandom Number

- ▶ A pseudorandom stream is one that is unpredictable without knowledge of the input key
- ▶ The output of the generator, called a keystream, is combined one byte at a time with the plaintext stream using the bitwise XOR operation.

Stream Cipher Properties

- [KUMA97] lists the following important design considerations for a stream cipher:
 1. The encryption sequence should have a large period. A pseudorandom number generator uses a function that produces a deterministic stream of bits that eventually repeats. The longer the period of repeat the more difficult it will be to do cryptanalysis.
 2. The keystream should approximate the properties of a true random number stream as close as possible, the more random-appearing the keystream is, the more randomized the ciphertext is, making cryptanalysis more difficult.
 3. To guard against brute-force attacks, the key needs to be sufficiently long. The same considerations as apply for block ciphers are valid here. Thus, with current technology, a key length of at least 128 bits is desirable.

RC4 Stream Cipher

- This is the most popular symmetric stream cipher
- Designed by Rivest for RSA Security
- Used in SSL/TLS (Secure Sockets Layer/Transport Layer Security) standards for secure communication between Web browsers and servers
- Used in WEP, part of the IEEE 802.11 wireless LAN standard
- RC4 was kept as a trade secret by RSA Inc but got anonymously posted on the Internet in 1994

RC4 algorithm

- Key length is variable: from 1 to 256 bytes
- Based on the key initialize a 256-byte state vector **S**: $S[0 \dots 255]$
 - At all times S contains a permutation of the numbers 0, 1, ..., 255
- For encryption and decryption a byte **k** is selected from S and the entries in S are permuted

RC4 – initialization of S

- Initially $S[i]=i$, $i=0,1,\dots,255$ and create a temporary vector T of length 256 – the key K is copied to T (if K has less than 256 bytes, repeat K as many times as necessary to fill T)
 - For $i=0$ to 255 do
 - $S[i]=i$;
 - $T[i]=K[i \bmod \text{keylen}]$
- Use T to produce the initial permutation of S
 - $j=0$;
 - For $i=0$ to 255 do
 - $j=(j+S[i]+T[i]) \bmod 256$;
 - Swap($S[i], S[j]$);
- Input key is never used after this initialization

RC4 Encryption

- ▶ encryption continues shuffling array values
- ▶ sum of shuffled pair selects "stream key" value from permutation
- ▶ XOR $S[t]$ with next byte of message to en/decrypt

$i = j = 0$

for each message byte M_i

$i = (i + 1) \pmod{256}$

$j = (j + S[i]) \pmod{256}$

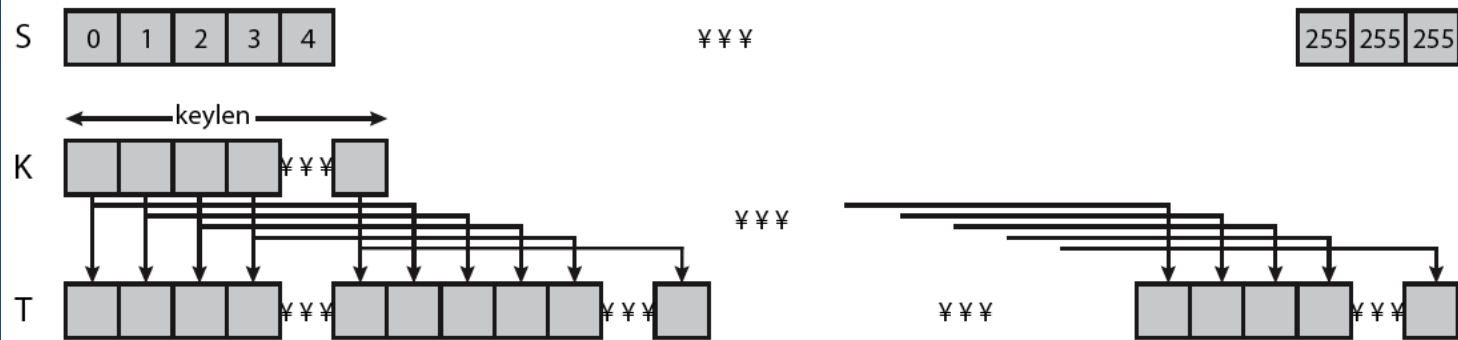
swap($S[i], S[j]$)

$t = (S[i] + S[j]) \pmod{256}$

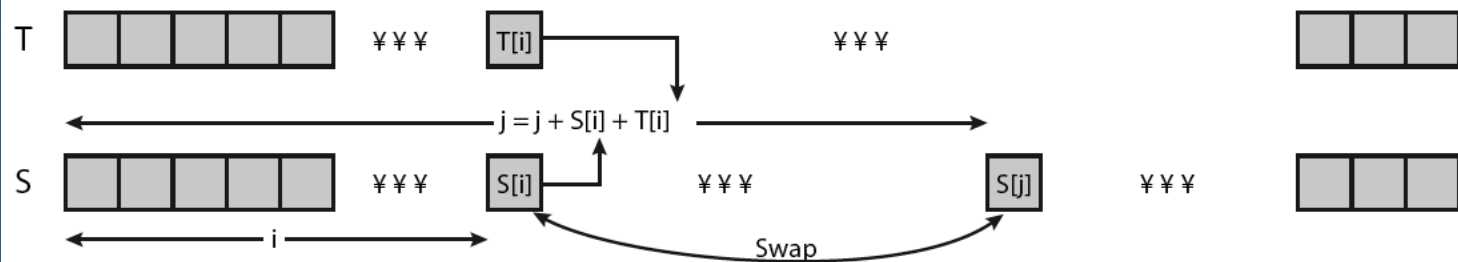
$K = S[t]$

$C_i = M_i \text{ XOR } K$

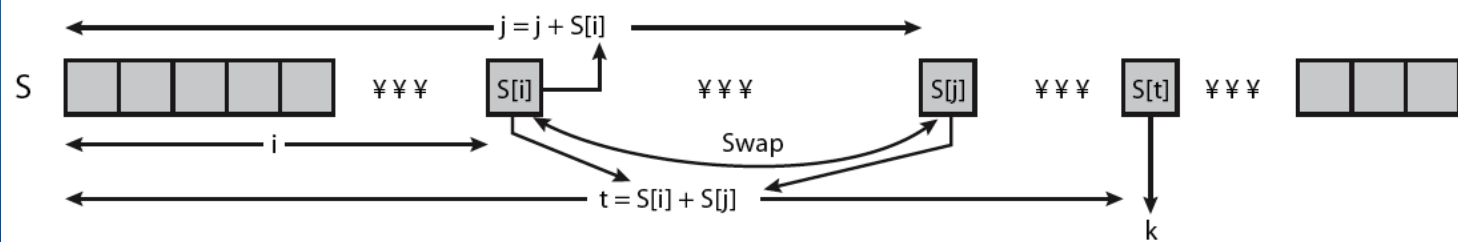
RC4 Overview



(a) Initial state of S and T



(b) Initial permutation of S



(c) Stream Generation

RC4 Algorithm

- ▶ Developed by RSA labs
- ▶ Symmetric
- ▶ Stream Cipher
 - ▶ Block Cipher that uses a different key for each block of plaintext in a stream.
 - ▶ WEP generally uses a counter to change the key.

RC4 Description

- ▶ Two main parts:
 - ▶ KSA (Key Scheduling Algorithm)
 - ▶ PRGA (Pseudo Random Generation Algorithm)
- ▶ Notation:
 - ▶ $S = \{0, 1, 2, \dots, N-1\}$ is the initial permutation
 - ▶ ℓ = length of key

RC4 Description

KSA(K)

Initialization:

For $i = 0 \dots N - 1$

$S[i] = i$

$j = 0$

Scrambling:

For $i = 0 \dots N - 1$

$j = j + S[i] + K[i \bmod \ell]$

$Swap(S[i], S[j])$

PRGA(K)

Initialization:

$i = 0$

$j = 0$

Generation loop:

$i = i + 1$

$j = j + S[i]$

$Swap(S[i], S[j])$

Output $z = S[S[i] + S[j]]$

RC4 Example

- ▶ Simple 4-byte example
- ▶ $S = \{0, 1, 2, 3\}$
- ▶ $K = \{1, 7, 1, 7\}$
- ▶ Set $i = j = 0$

KSA

First Iteration ($i = 0, j = 0, S = \{0, 1, 2, 3\}$):

$$j = (j + S[i] + K[i]) = (0 + 0 + 1) = 1$$

Swap $S[i]$ with $S[j]$: $S = \{1, 0, 2, 3\}$

Second Iteration ($i = 1, j = 1, S = \{1, 0, 2, 3\}$):

$$j = (j + S[i] + K[i]) = (1 + 0 + 7) = 0 \pmod{4}$$

Swap $S[i]$ with $S[j]$: $S = \{0, 1, 2, 3\}$

KSA

Third Iteration ($i = 2, j = 0, S = \{0, 1, 2, 3\}$):

$$j = (j + S[i] + K[i]) = (0 + 2 + 1) = 3$$

Swap $S[i]$ with $S[j]$: $S = \{0, 1, 3, 2\}$

Fourth Iteration ($i = 3, j = 3, S = \{0, 1, 3, 2\}$):

$$j = (j + S[i] + K[i]) = (3 + 2 + 7) = 0 \pmod{4}$$

Swap $S[i]$ with $S[j]$: $S = \{2, 1, 3, 0\}$

PRGA

Reset $i = j = 0$, Recall $S = \{2, 1, 3, 0\}$

$i = i + 1 = 1$

$j = j + S[i] = 0 + 1 = 1$

Swap $S[i]$ and $S[j]$: $S = \{2, 1, 3, 0\}$

Output $z = S[S[i] + S[j]] = S[2] = 3$



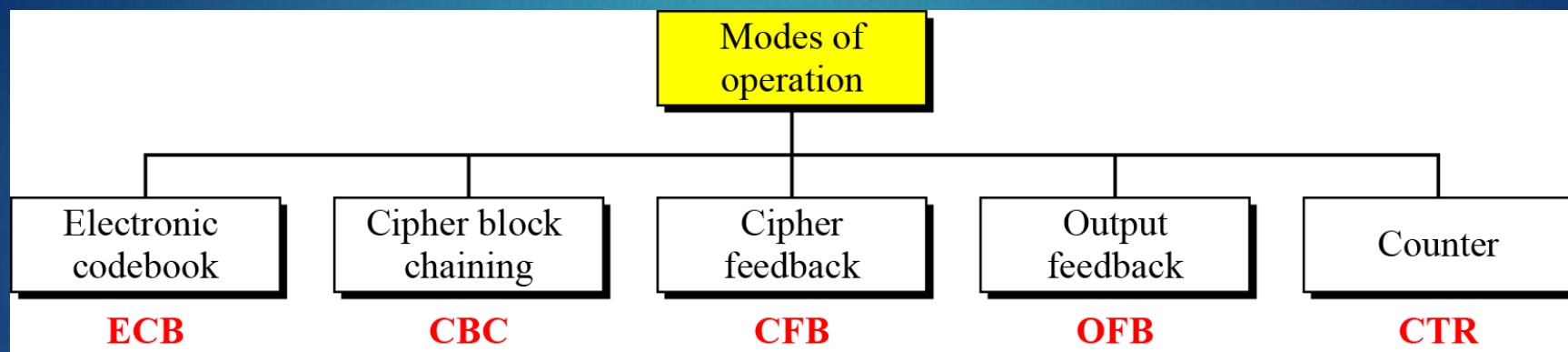
Chapter 6-Cipher Block Modes of Operation

8-1 USE OF MODERN BLOCK CIPHERS

Symmetric-key encipherment can be done using modern block ciphers. Modes of operation have been devised to encipher text of any size employing either DES or AES.

Topics discussed in this section:

- 8.1.1 Electronic Codebook (ECB) Mode
- 8.1.2 Cipher Block Chaining (CBC) Mode
- 8.1.3 Cipher Feedback (CFB) Mode
- 8.1.4 Output Feedback (OFB) Mode
- 8.1.5 Counter (CTR) Mode

Figure 8.1 *Modes of operation*

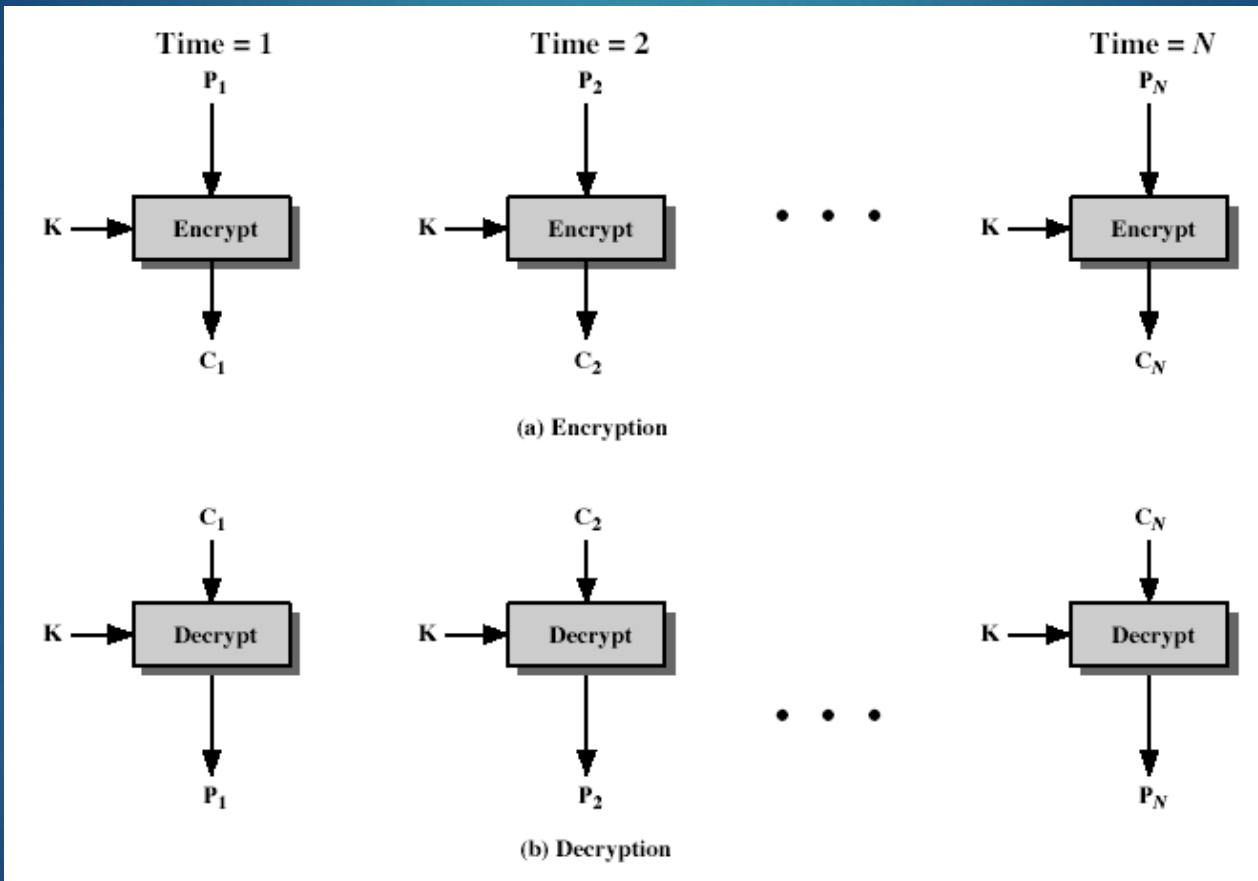
Electronic Codebook Book (ECB)

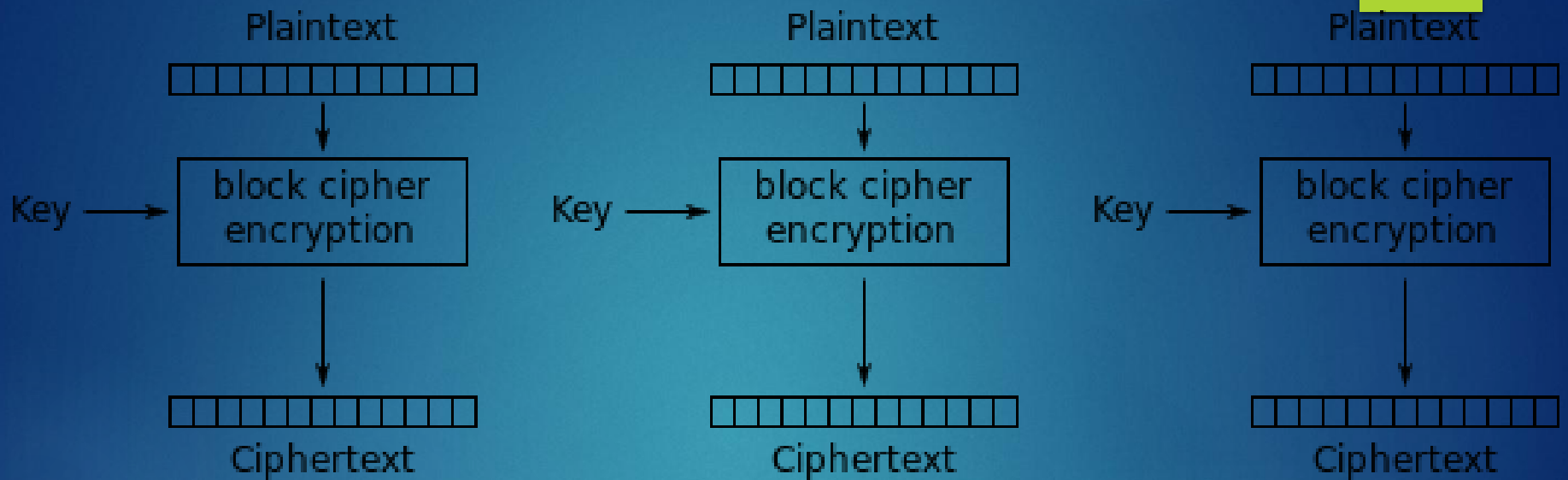
- ▶ message is broken into independent blocks which are encrypted
- ▶ each block is a value which is substituted, like a codebook, hence name
- ▶ each block is encoded independently of the other blocks

$$C_i = \text{DES}_{K1}(P_i)$$

- ▶ uses: secure transmission of single values

Electronic Codebook Book (ECB)





Electronic Codebook (ECB) mode encryption

Advantages and Limitations of ECB

- ▶ repetitions in message may show in ciphertext
 - ▶ if aligned with message block
 - ▶ particularly with data such graphics
 - ▶ or with messages that change very little, which become a code-book analysis problem
- ▶ weakness due to encrypted message blocks being independent
- ▶ main use is sending a few blocks of data

Cipher Block Chaining (CBC)

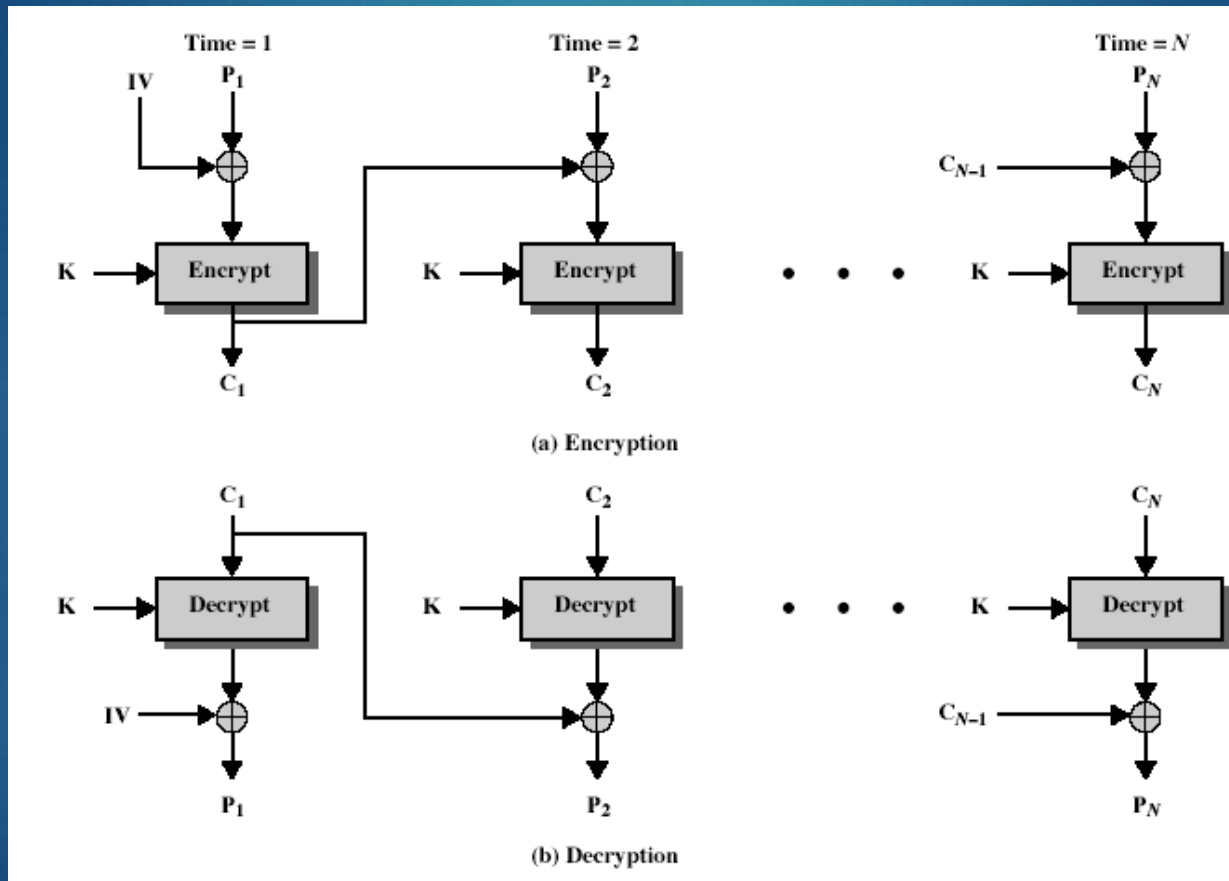
- ▶ message is broken into blocks
- ▶ but these are linked together in the encryption operation
- ▶ each previous cipher blocks is chained with current plaintext block, hence name
- ▶ use Initial Vector (IV) to start process

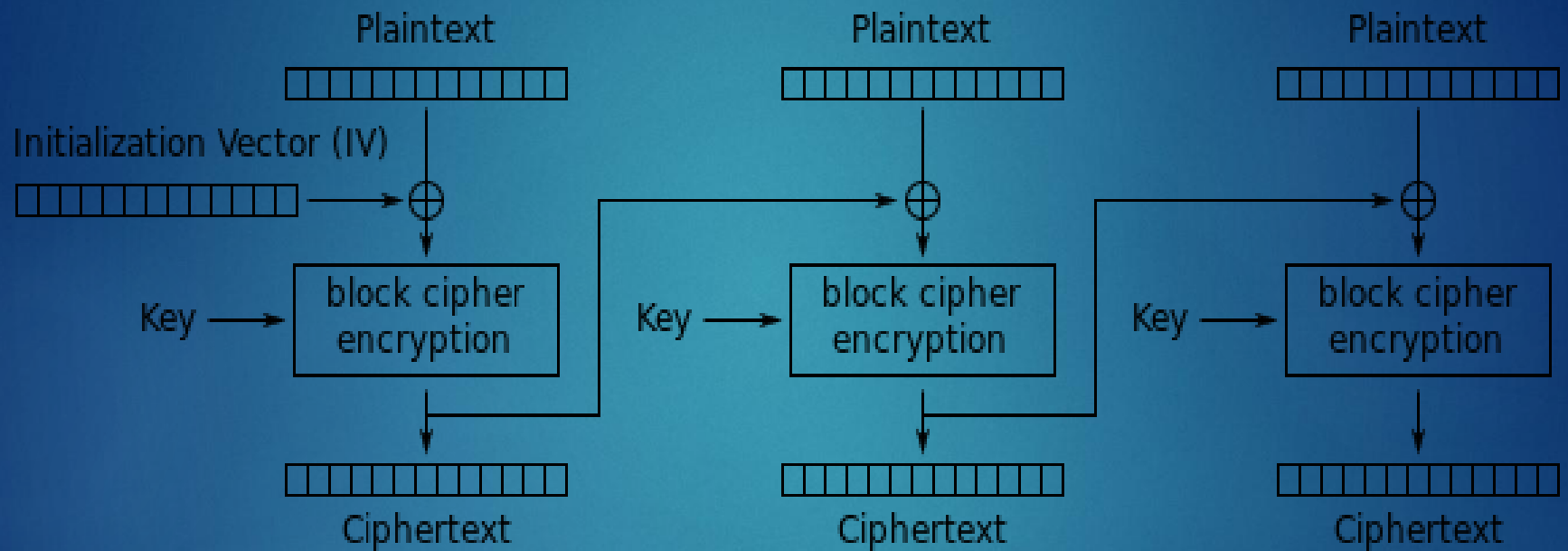
$$C_i = \text{DES}_{K1}(P_i \text{ XOR } C_{i-1})$$

$$C_{-1} = \text{IV}$$

- ▶ uses: bulk data encryption, authentication

Cipher Block Chaining (CBC)





Cipher Block Chaining (CBC) mode encryption

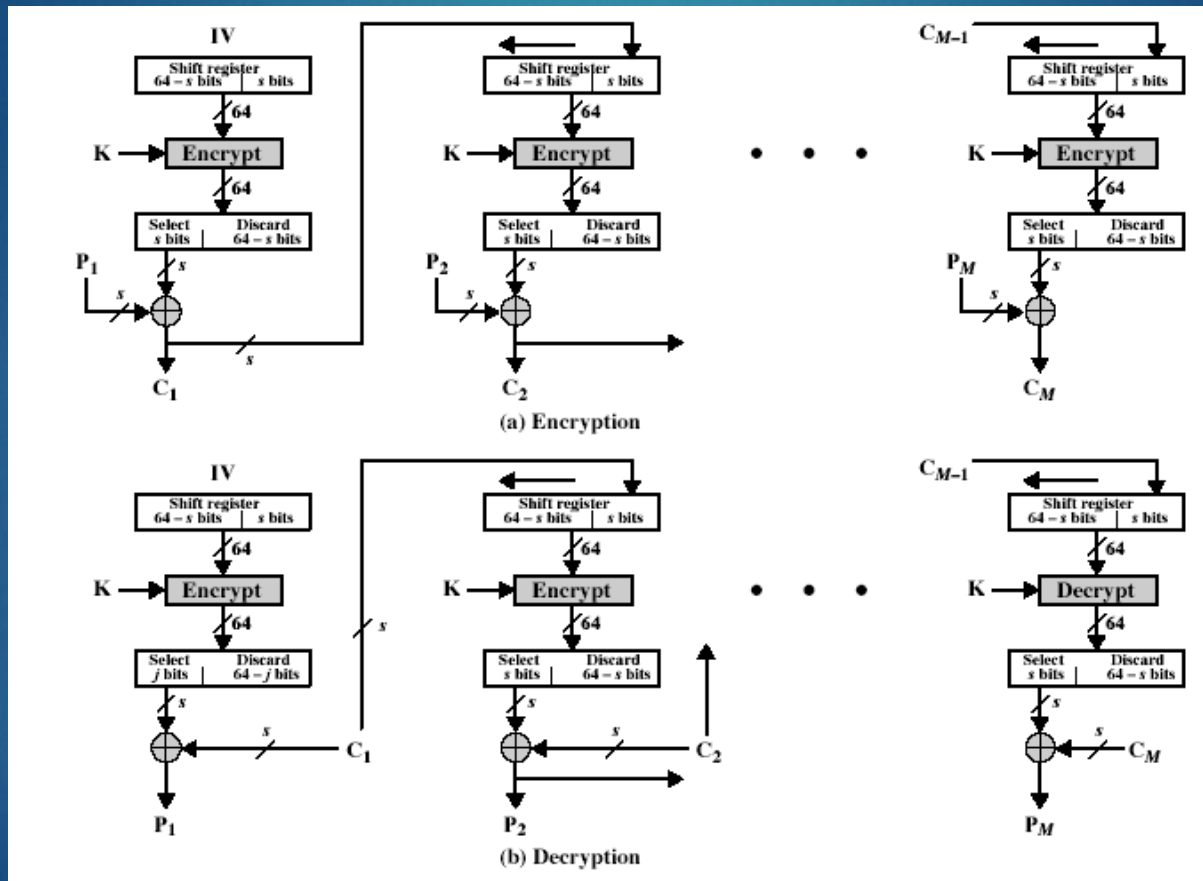
Advantages and Limitations of CBC

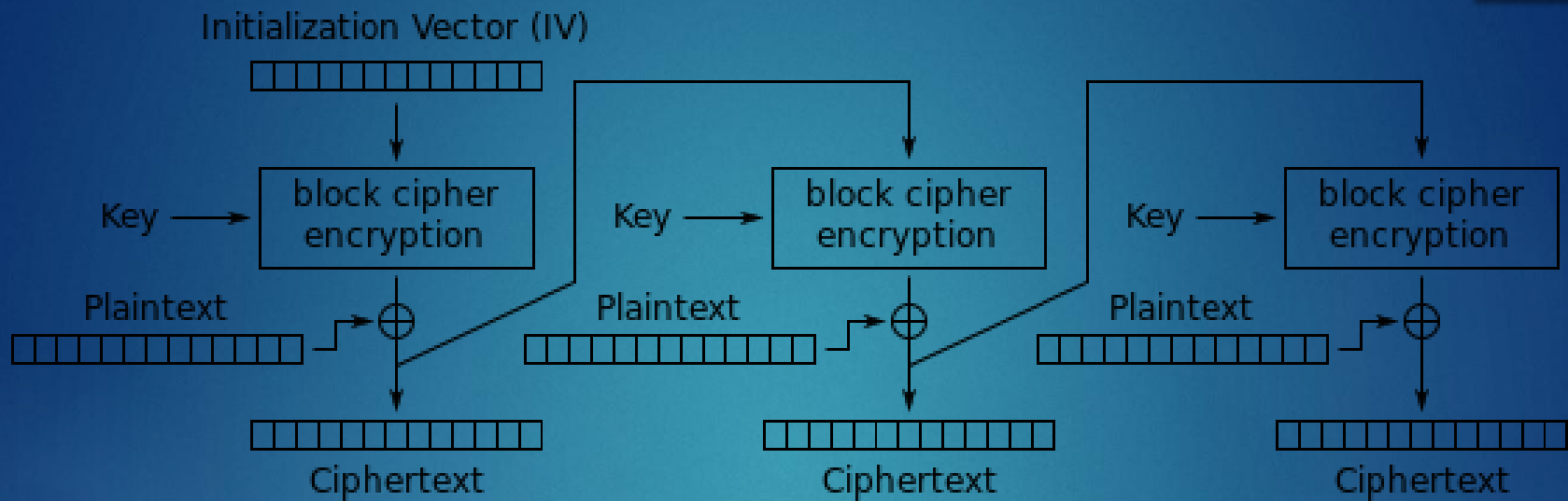
- ▶ each ciphertext block depends on **all** message blocks
- ▶ thus a change in the message affects all ciphertext blocks after the change as well as the original block
- ▶ need **Initial Value (IV)** known to sender & receiver
 - ▶ however if IV is sent in the clear, an attacker can change bits of the first block, and change IV to compensate
 - ▶ hence either IV must be a fixed value or it must be sent encrypted in ECB mode before rest of message
- ▶ at end of message, handle possible last short block
 - ▶ by padding either with known non-data value (eg nulls)
 - ▶ or pad last block with count of pad size
 - ▶ eg. [b1 b2 b3 0 0 0 0 5] <- 3 data bytes, then 5 bytes pad+count

Cipher FeedBack (CFB)

- ▶ message is treated as a stream of bits
- ▶ added to the output of the block cipher
- ▶ result is feed back for next stage (hence name)
- ▶ standard allows any number of bit (1,8 or 64 or whatever) to be feed back
 - ▶ denoted CFB-1, CFB-8, CFB-64 etc
- ▶ is most efficient to use all 64 bits (CFB-64)
$$C_i = P_i \text{ XOR } \text{DES}_{K1}(C_{i-1})$$
$$C_{-1} = \text{IV}$$
- ▶ uses: stream data encryption, authentication

Cipher FeedBack (CFB)





Cipher Feedback (CFB) mode encryption

Advantages and Limitations of CFB

- ▶ appropriate when data arrives in bits/bytes
- ▶ most common stream mode
- ▶ limitation is need to stall while do block encryption after every n-bits
- ▶ note that the block cipher is used in **encryption** mode at **both** ends
- ▶ errors propagate for several blocks after the error

Output FeedBack (OFB)

- message is treated as a stream of bits
- output of cipher is added to message
- output is then feed back (hence name)
- feedback is independent of message
- can be computed in advance

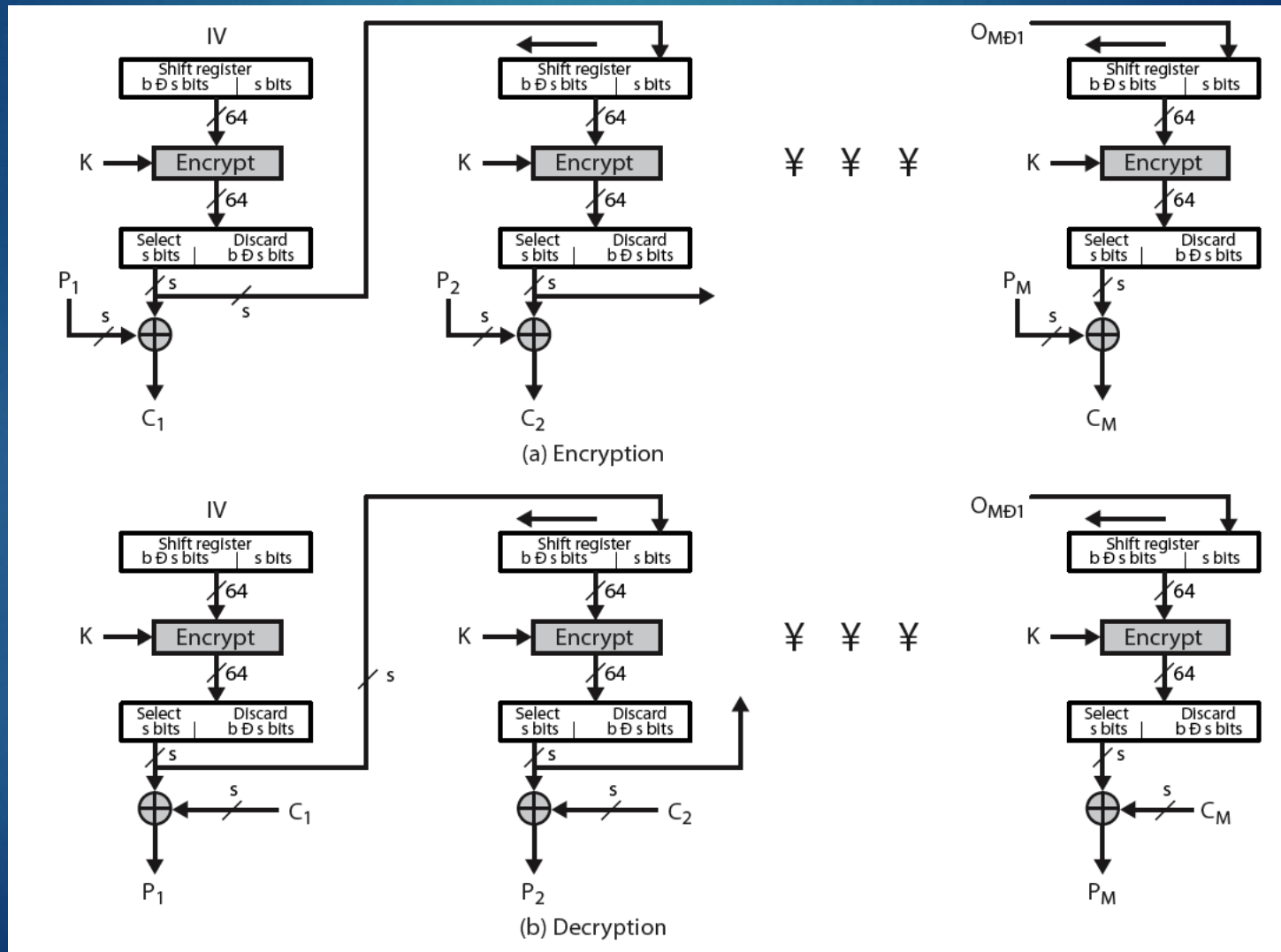
$$C_i = P_i \text{ XOR } O_i$$

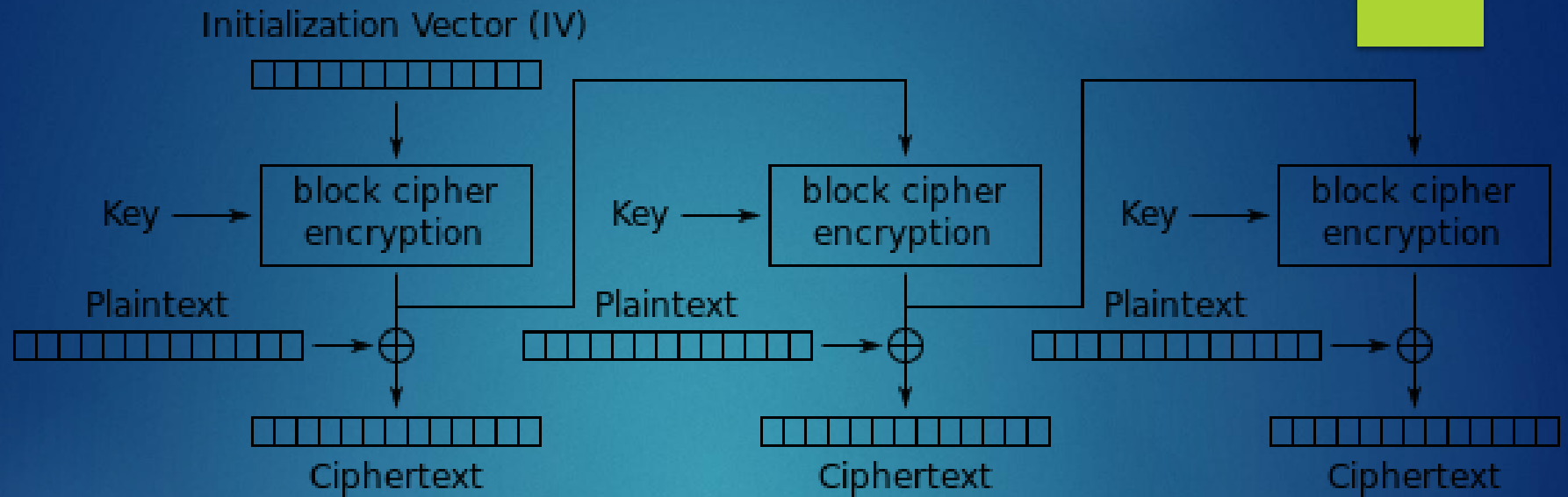
$$O_i = \text{DES}_{K1}(O_{i-1})$$

$$O_0 = \text{IV}$$

- uses: stream encryption on noisy channels

Output FeedBack (OFB)





Output Feedback (OFB) mode encryption

Advantages and Limitations of OFB

- bit errors do not propagate
- more vulnerable to message stream modification
- Just like the CFB a variation of a Vernam cipher
 - hence must **never** reuse the same sequence (key+IV)
- sender & receiver must remain in sync

Counter (CTR)

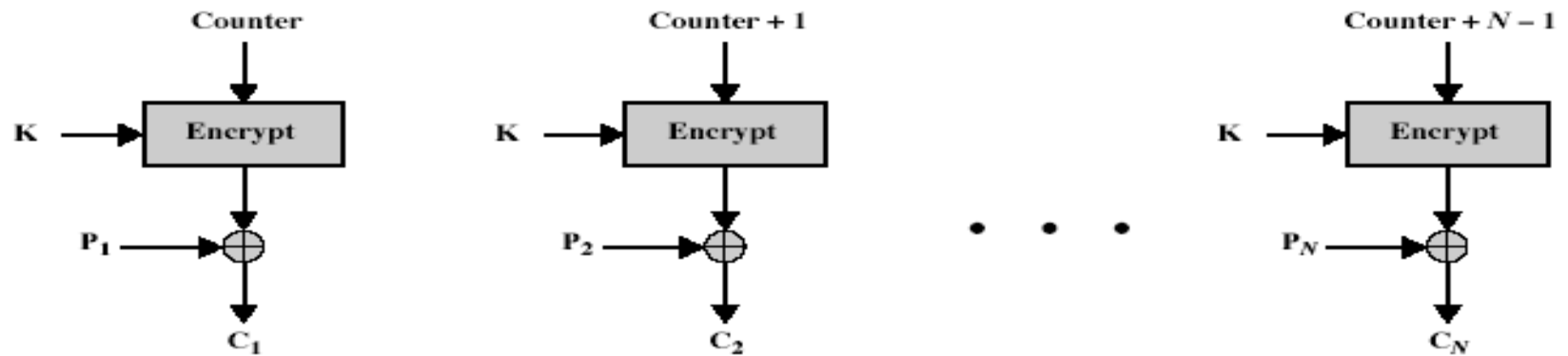
- a “new” mode, though proposed early on
- similar to OFB but encrypts counter value rather than any feedback value
- must have a different key & counter value for every plaintext block (never reused)

$$C_i = P_i \text{ XOR } O_i$$

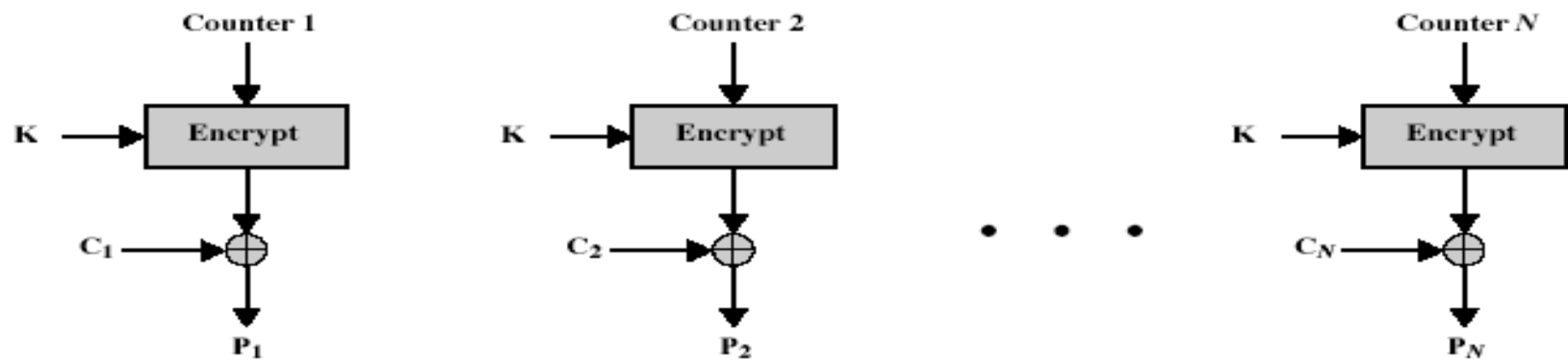
$$O_i = \text{DES}_{K1}(i)$$

- uses: high-speed network encryptions

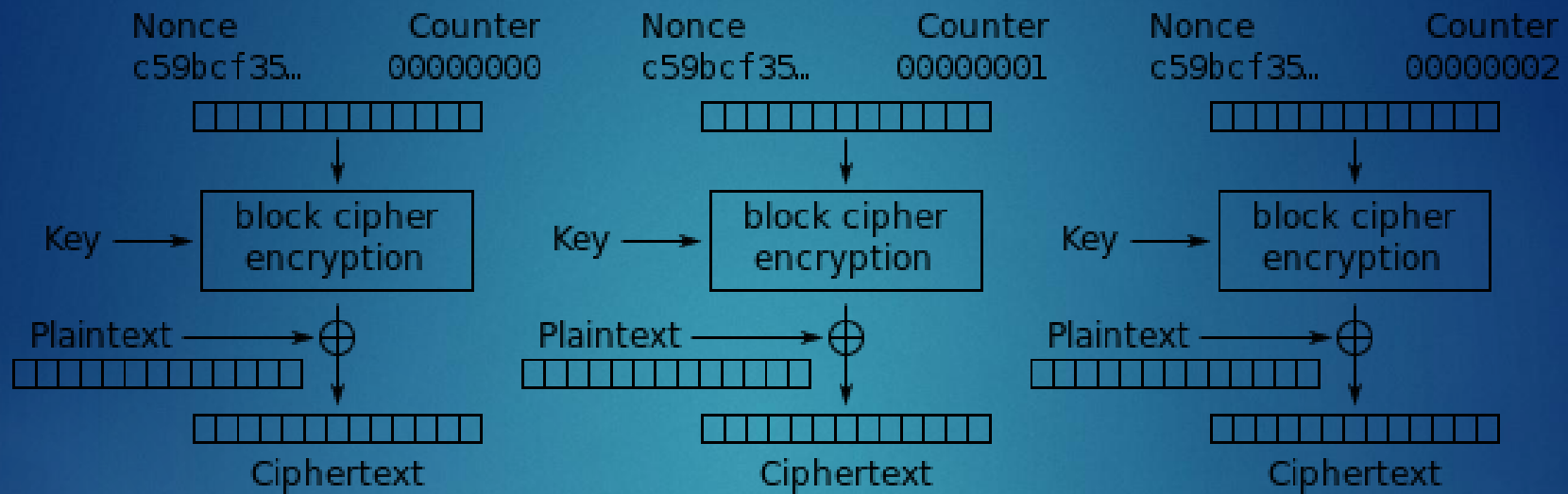
Counter (CTR)



(a) Encryption



(b) Decryption



Counter (CTR) mode encryption

Advantages and Limitations of CTR

- efficiency
 - can do parallel encryptions in h/w or s/w
 - can preprocess in advance of need
 - good for bursty high speed links
- random access to encrypted data blocks
- provable security (good as other modes)
- but must ensure never reuse key/counter values, otherwise could break (cf OFB)

CBC and CTR comparison

64

CBC	CTR
Padding needed	No padding
No parallel processing	Parallel processing
Separate encryption and decryption functions	Encryption function alone is enough
Random IV or a nonce	Unique nonce
Nonce reuse leaks some information about initial plaintext block	Nonce reuse will leak information about the entire message